

苏剑林研究专题 11: Linear Attention -> DeltaNet -> KDA -> Kimi Linear

从 Kernel Trick、固定状态到高效混合架构

2026-08-14

Contents

摘要	2
1. 标准 Attention 为什么是平方复杂度	2
2. Kernel Linear Attention	2
3. Attention 其实可以视为 RNN	2
4. 简单累加为什么不够	3
5. Delta Rule	3
6. Gating: 状态需要可控遗忘	3
7. 从 Gated DeltaNet 到 KDA	3
8. DPLR / 结构化递推	3
9. Hybrid Attention 为什么现实	4
10. 训练并行: chunkwise algorithm	4
11. 与 KV-cache 的系统比较	4
12. Gotchas	4
12.1 只说 $O(L)$ 而忽略 d^2	4
12.2 训练 FLOPs 低不等于 wall-clock 快	4
12.3 固定状态的“无限上下文”不是无损记忆	4
13. Know-how: 如何评价 Linear Attention	4
14. Know-why	5
复习清单	5
参考资料 (精选)	5

系列定位。本报告属于“苏剑林研究专题”系列。报告将三类内容明确区分: (1) 苏剑林本人署名论文/项目; (2) 其在“科学空间”中长期公开推导与分析的研究性文章; (3) 其参与的团队论文 (例如 Kimi 系列)。对于团队工作, 不把公开论文无法确认的模块主导权归因给某一位作者。

阅读方法。不建议只记结论。每一节都尽量按“问题 -> 数学约束 -> 结构 -> 工程实现 -> 边界条件”展开, 并单独列出 gotchas、know-how 与 know-why。

摘要

Linear Attention 的目标通常被概括为“把 $O(L^2)$ 变成 $O(L)$ ”，但真正困难的是：softmax Attention 不只是复杂度高，它还提供强大的内容寻址和动态归一化。苏剑林长期从“Attention 与 RNN 的统一”视角分析这一问题，并参与 Kimi Linear/KDA 团队研究。本专题从 kernel trick 出发，推到 recurrent state、Delta Rule、gating 与 Kimi Delta Attention。

1. 标准 Attention 为什么是平方复杂度

$$Y = \text{softmax}(QK^\top)V.$$

$QK^\top$ 是 $L \times L$ 矩阵，因此时间/中间空间通常至少 $O(L^2)$ 。

自回归 decode 可以用 KV-cache 避免重复计算，但状态大小仍随 L 线性增长。

2. Kernel Linear Attention

如果把相似度写成可分解 kernel:

$$\text{sim}(q, k) = \phi(q)^\top \phi(k),$$

那么

$$y_i = \frac{\sum_{j \leq i} \phi(q_i)^\top \phi(k_j) v_j}{\sum_{j \leq i} \phi(q_i)^\top \phi(k_j)}.$$

利用结合律:

$$S_i = \sum_{j \leq i} \phi(k_j) v_j^\top, \quad z_i = \sum_{j \leq i} \phi(k_j),$$

则

$$y_i = \frac{\phi(q_i)^\top S_i}{\phi(q_i)^\top z_i}.$$

状态 S_i, z_i 可以递推更新，不需要存整个历史。

3. Attention 其实可以视为 RNN

上式已经是 recurrent state:

$$S_i = S_{i-1} + \phi(k_i) v_i^\top.$$

因此 linear attention 本质上把“保存所有历史 KV”改成“在线压缩到固定大小状态”。

这带来一个根本 trade-off: 固定状态无法无损记住任意长的所有 token，必须学习什么信息值得保留、覆盖或遗忘。

4. 简单累加为什么不够

若永远

$$S_t = S_{t-1} + k_t v_t^\top,$$

旧信息不断叠加，状态会饱和；当多个 key 相似时，新旧 value 互相干扰。需要一种“写入前先擦除冲突记忆”的规则。

5. Delta Rule

Delta Rule 的一个典型形式：先根据当前状态预测 value

$$\hat{v}_t = S_{t-1}^\top k_t,$$

误差为

$$e_t = v_t - \hat{v}_t,$$

再更新

$$S_t = S_{t-1} + \beta_t k_t e_t^\top.$$

如果 key 已经存过类似映射，只写入“残差”；如果新信息与旧状态冲突，会主动纠正。这比纯累加更像在线 associative memory 学习。

6. Gating：状态需要可控遗忘

很多现代 linear/recurrent attention 引入 gate：

$$S_t = \alpha_t \odot S_{t-1} + \text{write}_t.$$

α_t 决定不同通道保留多少历史。没有 gating，固定状态必须永远承担全部历史；有 gating 后可以形成动态时间尺度。

7. 从 Gated DeltaNet 到 KDA

Kimi Linear 中的 Kimi Delta Attention (KDA) 进一步发展 delta-rule 路线, 强调更表达性的 channel-wise gating, 并设计适合 GPU 的 chunkwise 训练算法。

一个关键目标是同时满足：

- 训练阶段可并行，不能真的逐 token 慢 RNN；
- 推理阶段状态固定，不随 context 增长；
- 表达能力接近 full attention。

8. DPLR / 结构化递推

高效 recurrent 模型经常把状态转移写成 diagonal + low-rank 或其他结构, 原因是一般 dense recurrence 很难并行。DPLR 让状态更新既有通道独立的高效部分，又有低秩交互提供全局耦合。

它体现出一个典型系统设计原则：数学结构必须允许 scan/chunk 并行，才可能成为 Transformer 的真正替代。

9. Hybrid Attention 为什么现实

Kimi Linear 等架构不是必须“纯 linear”。周期性插入 full/MLA attention 可以提供精确全局内容寻址，而大量层使用 KDA 维持固定状态成本。

可以把它理解为：

cheap recurrent memory + occasional exact retrieval.

这类似 cache hierarchy，而不是非黑即白地选择 RNN 或 Attention。

10. 训练并行：chunkwise algorithm

递推式

$$S_t = f(S_{t-1}, x_t)$$

看似必须串行。高性能实现会把序列分块，在块内用矩阵化公式并行计算，再在块间传递 compact state。目标是把大部分计算转成 GEMM/scan，而不是 Python-level loop。

11. 与 KV-cache 的系统比较

Full Attention：

- 状态： $O(Ld)$ KV cache；
- 检索：任意历史 token 可直接内容寻址。

Linear Attention：

- 状态： $O(d^2)$ 或结构化固定状态；
- 检索：历史被压缩，可能发生干扰。

因此长到某个阈值后，linear attention 的 memory 优势会越来越明显，但短 context 下 full attention 可能仍更简单高效。

12. Gotchas

12.1 只说 $O(L)$ 而忽略 d^2

某些 linear attention state 是 $d_h \times d_v$ 矩阵。head dim 很大时常数并不小。

12.2 训练 FLOPs 低不等于 wall-clock 快

如果 kernel 无法充分利用 Tensor Core，理论复杂度更低也可能比 FlashAttention 慢。

12.3 固定状态的“无限上下文”不是无损记忆

状态大小固定意味着信息必须压缩。无限长度只是计算可以继续，并不表示模型可精确记住无限历史。

13. Know-how：如何评价 Linear Attention

不要只做 perplexity。至少同时测：

- associative recall;
- long-range copy;
- needle retrieval;
- 长文问答;
- state size;
- prefill/decode throughput;

- context length 扩展后的性能退化曲线。

并与同等参数量、同等 FLOPs 的 full/hybrid attention 对比。

14. Know-why

Linear Attention 研究的核心并不是复杂度符号，而是**如何把一段任意长的历史压缩为有限状态，同时仍保留对未来最有用的信息**。从这个角度看，它与经典 RNN、在线学习、associative memory 和现代 Attention 实际上属于同一个问题族。

复习清单

建议在完成本专题后，能够回答下面四类问题：

1. **定义题**：核心对象、公式和变量分别是什么？
2. **推导题**：为什么这种结构能够解决原问题？关键等式如何得到？
3. **工程题**：实现时真正容易出错的地方是什么？哪些超参数决定行为？
4. **迁移题**：如果数据规模、上下文长度、模型宽度或训练预算变化，原结论还成立吗？

参考资料（精选）

- 科学空间：将 Attention 视为平方复杂度的 RNN —<https://spaces.ac.cn/archives/10017>
- 科学空间：线性注意力简史—<https://spaces.ac.cn/archives/11033>
- Kimi Linear: An Expressive, Efficient Attention Architecture —<https://arxiv.org/abs/2510.26692>

版本说明：2026-08-14。本文以公开论文、公开项目与“科学空间”公开文章所形成的研究脉络为基础，重点用于技术学习与研究复盘，而非作者个人传记。